八算子定义与优化讲解

面向初学者的汇报讲解，整理于2026年10月29日。本文解释项目累计有效交付的方法，不表示八个算子都在本周新增，也不表示全部由独立Agent自动取得。

先理解：kernel启动、program和融合

kernel启动（kernel launch），指主机端向NPU提交一次kernel执行。主机负责准备参数并提交，设备将工作分配给计算核。

一次kernel启动里可以有很多Triton program，每个program处理一部分数据。

- 融合多个kernel：可能减少kernel启动次数，也可能减少中间结果和重复读写。
- 扩大分块：通常减少同一次启动中的program数量，不一定减少kernel启动次数。
- 优化启动路径：减少主机端准备参数、地址处理、缓存查询与提交等开销。

1. value_selecting_reduction_backward：把梯度传回选中的位置

定义

max/min等前向操作选出了一个元素。反向传播时，这个算子把梯度放回被选中的位置，其他位置填零。它接收前向保存的位置，不需要重新寻找最大值。

例子：

```
前向输入：[2, 7, 4]
选中位置：1（下标从0开始）
上游梯度：3
输出梯度：[0, 3, 0]
```

性能优化

将“输出补零”和“梯度回填”合并处理：选中位置写梯度，其余位置写零，直接生成最终结果。减少独立操作以及先清零、再覆盖的重复写入。

正确性修复

按张量真实的shape/stride计算地址，正确处理转置、切片等非连续布局。逻辑相邻元素未必对应相邻内存地址，不能总是假设地址每次加1。

汇报一句话：融合补零与梯度回填提升性能，正确处理非连续布局补齐正确性。

结果：相对PyTorch为1.635×，正式评测96/96通过。

2. scatter_add：按索引把数累加到指定位置

定义

每个输入元素携带目标位置，将它加到对应输出位置；多个元素可以累加到同一位置。

```
待累加数：[2, 3, 4]
目标位置：[0, 1, 0]
初始输出：[0, 0]
最终输出：[6, 3]
```

性能优化

采用排序、片上查找和按目标分段累加的实现组织，把相同目标的贡献集中处理，减少对输出位置反复更新和竞争。

正确性修复

显式按目标分段计算，避免不同目标间的浮点数值干扰。旧候选用整行前缀和相减提取分组结果，在较大动态范围下，大数可能吞掉小数；分组独立计算避免这类跨组误差。

汇报一句话：按目标组织与分段累加提高效率，避免跨目标数值干扰。

结果：相对PyTorch为1.900×，正式评测18/18及补充动态范围等检查通过。

3. uniform：向张量填入均匀分布随机数

定义

在指定区间内生成均匀分布的随机数并填入张量，例如在[0, 1)内生成[0.12, 0.85, 0.43, ...]。

实际使用伪随机算法。给定种子、计数器和状态，应按约定生成可复现的随机序列。

性能优化

优化随机状态读取与启动路径，减少生成随机数之前的准备开销。

正确性修复

修复Philox随机数算法中的无符号移位、计数器进位等整数运算，并核对随机状态推进。“看起来分布均匀”不足以证明算法符合要求。

汇报一句话：减少状态读取与启动开销提高性能，修复整数运算保证随机算法正确。

结果：相对PyTorch为1.325×，正式18/18、标准Philox逐位及随机状态检查通过。

4. rad2deg_：将弧度转成角度并原地写回

定义

对每个元素执行x <- x * 180/pi。例如[0, pi/2, pi]变成[0, 90, 180]。

名称末尾的下划线表示直接修改原张量，而不是只返回一个新的结果张量。

性能优化

调整分块，让每个program处理合适数量的元素，并优化启动路径。计算本身简单，任务组织与主机提交可能占比较大。

正确性修复

保持原地修改与别名语义，正确处理非连续输入。把非连续输入复制成连续张量后只修改副本，并不满足原地接口。

汇报一句话：调整分块和启动路径提高性能，修复原地修改行为保证接口正确。

结果：相对PyTorch为1.117×，正式36/36及补充检查通过。

5. upsample_bilinear2d：双线性插值放大图像或特征图

定义

将二维图像或特征图调整到更大的尺寸。每个输出位置通常根据输入中邻近四个位置的数值与距离加权计算，边界位置按接口规则处理。

性能优化

连续读取相邻输入行，一次计算两行输出，让邻近输出利用已经读取的数据，减少重复读取与寻址工作。

正确性修复

明确每个输出位置归哪个分块负责，修复分块交界处重复写入问题。

汇报一句话：利用相邻输出的数据复用提高性能，明确分块写入范围保证正确性。

结果：相对PyTorch为0.915×，正式24/24及84项补充检查通过。超过0.8×验收线，但仍略慢于PyTorch。

6. log10_：逐元素求以10为底的对数

定义

对每个元素执行x <- log10(x)，例如[1, 10, 100, 1000]变成[0, 1, 2, 3]。结果原地写回，并按参考语义处理零、负数、无穷和NaN等特殊值。

性能优化

减少中间类型转换，并优化启动路径。减少不必要的转换可以降低额外指令与缓冲成本。

正确性修复

修复高维张量的shape/stride寻址，避免高维非连续输入漏算或写错位置，保留特殊值与原地语义。

汇报一句话：减少中间转换与启动开销提高性能，修复高维寻址保证正确性。

结果：相对PyTorch为0.875×，正式36/36及补充检查通过。超过0.8×验收线，但仍慢于PyTorch。

7. selu_：逐元素执行SELU激活函数

定义

正数做线性缩放，负数做指数变换：

```
x > 0：y = lambda * x
x <= 0：y = lambda * alpha * (exp(x) - 1)
```

alpha和lambda是SELU规定的常数。结果原地写回，常用于神经网络激活。

性能优化

调整分块与中间计算精度，并优化启动。分块太小会产生较多program；分块过大可能占用过多片上缓冲。精度安排要在满足误差要求的前提下控制转换、计算与缓冲成本。

正确性要求

保留原地语义、布局处理和特殊值行为。本项主要强调性能调优，不额外声称每个改动都修复了一个已发现的正确性问题。

汇报一句话：选择合适的分块与精度安排，减少执行开销，同时保持完整正确性。

结果：相对PyTorch为0.837×，正式36/36及补充检查通过。另批次测值不与本批混合。

8. gelu_and_mul：一部分做GELU，再与另一部分相乘

定义

把输入分成a、b两部分，计算y = GELU(a) * b。GELU是神经网络激活函数，激活与乘法组成常见门控结构。

性能优化

连续二维分块处理，简化动态索引和地址计算，优化启动路径。原实现已是融合计算，不把提升描述为“首次将GELU和乘法融合”。

正确性修复与参考公式

这里的“本机参考”就是KGS评测使用的本机PyTorch NPU参考。

GELU常见两种形式：

```
精确形式：GELU(x) = 0.5 * x * (1 + erf(x / sqrt(2)))
tanh近似：GELU(x) ≈ 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715*x^3)))
```

本机固定软件栈的探针发现：在两组各2^20个FP32样本上，F.gelu(approximate='none')与'tanh'逐位相同，接近CPU的tanh公式。候选对齐了本机实际评测参考，FP32补充检查随后通过。

这个结论针对本机固定PyTorch/torch_npu软件栈及已测样本，不表示所有平台的PyTorch都会将两种形式算成相同结果。

汇报一句话：连续分块和简化索引提高性能，对齐本机PyTorch参考公式保证数值一致。

结果：相对PyTorch为2.271×，正式19/19及补充检查通过。

汇报口径与证据入口

- 上述数值为KGS完整调用计时相对等价PyTorch参考的加速比；超过0.8×为验收达标，超过1×才表示比PyTorch快。
- 正确性不是逐渐“提高精度分数”，而是修复了特定布局、语义或数值问题，并通过相应测试。
- 方法汇总包含KG/HG-KDA、CC/DeepSeek与Codex的实际分析、生成和修复贡献，不归为纯原生KG独立成果。
- 当前有效结果见 operators/会议10算子最终验收-20261026.md、operators/夜间连续优化交付-20261023.md、operators/两算子收尾交付-20261024.md。
- GELU参考探针说明见 operators/夜间连续优化交付-20261023.md 的“关键机制”相关段落。
- 项目统一口径见 docs/实验归属与汇报口径.md。